

ROI - MVP1 Build Plan

Restaurant Operational Intelligence . Release 1 (web only) . Rails 8 + Hotwire monolith . Rebalanced to an exact 50/50 split between Tanner and Kedi

WORKLOAD SPLIT (now exactly even)	
Tanner	27 tasks owned outright . 42.75 estimated days . Engine, data backbone, realtime infra, pacing and cut logic, the Host backend.
Kedi	27 tasks owned outright . 42.75 estimated days . All frontend and views (Host views included), ROI Internal onboarding, Manager, analytics.
Shared	4 tasks . 7 days . Split evenly, so 3.5 days each. With shared added in, each dev carries 46.25 days, exactly half of 92.5.
See the Workload Split tab	Full per sprint and total breakdown proving the 50/50 by both task count and estimated days.
PHASES / SPRINTS AT A GLANCE	
Phase 0: Foundation (Sprint 0)	Shared base: Rails app, database, realtime, login, core models, fake event source. Split by component between both devs.
Phase 1: Onboarding + Engine Core (Sprint 1)	Kedi builds ROI Internal (incl. drag and drop floor plan). Tanner builds the deterministic rule engine.
Phase 2: Scoring + Floor Views (Sprint 2)	Tanner adds heuristic scoring. Kedi builds the Host floor and seating views and the Manager live view.
Phase 3: Pacing, Cuts, Rush (Sprint 3)	Tanner builds pacing, cut and rush logic. Kedi builds the rush toggle and cut cost views.
Phase 4: Analytics + Pilot (Sprint 4)	Analytics, no show tracking, the decision audit view, hardening, and shadow mode pilot readiness.
WHAT CHANGED IN THE REBALANCE	
Host frontend moved to Kedi	The Host views (live floor, greeter check in, seating recommendation surface, rush toggle, decision audit view) are now Kedi's. The engine and realtime infra behind them stay Tanner's.
A few tasks were split	H-01, H-03 and I-03 were each split into two sub tasks so the work divides cleanly and the counts land even.
Engine stays with Tanner	The deterministic seating engine, pacing governor, cut logic and VIP holds remain Tanner's. None of that moved.
OWNER LEGEND	
Tanner	Engine and backend (the high risk algorithmic core, the data backbone, realtime infra).
Kedi	Frontend and views, ROI Internal onboarding, Manager surface, analytics.
Shared	Done jointly or with an explicit written contract. Integration seams that span both domains.

Sprint 0: Shared Foundation (split by component)

Goal: Stand up the Rails 8 monolith, schema, realtime plumbing, auth and multi tenancy, core domain models, and the synthetic event source. Both devs are blocked on rr system and own models in parallel, then begins ROI Internal location setup (I-01) the moment the config models land.

Phase / Sprint	Task ID	Epic	Task (bullet)	What it means + why we do it
Phase 0: Foundation (Sprint 0)	F-01	Repo & tooling	• Initialize Rails 8.1.x app (Ruby 3.3+) with Postgres adapter and Hotwire default.	Create the empty Rails 8 project that everything else is built inside. We do it first because nothing can start until this container exists.
Phase 0: Foundation (Sprint 0)	F-02	Repo & tooling	• Set schema format to :sql (structure.sql) so Postgres partitioning and CHECK constraints survive schema dumps.	Tell Rails to record the database shape as raw SQL instead of its simpler Ruby format. We do it now because our events table needs Postgres partitioning the Ruby format cannot capture.
Phase 0: Foundation (Sprint 0)	F-03	Local DB env	• docker-compose.yml with a Postgres 15 service plus a documented native install path, and a .env.example.	Give each dev a local database they can run in Docker or as a desktop install. We do it so both of you can work on your own copy, set up the way you each prefer.
Phase 0: Foundation (Sprint 0)	F-04	Local DB env	• Migration workflow doc plus a seed script that builds one demo location.	Write the agreed steps for making a database change, committing it, and applying it on pull, plus test data. We do it to keep both local databases in sync.
Phase 0: Foundation (Sprint 0)	F-05	CI / quality	• GitHub Actions running migrations and tests on every pull request, with RuboCop and Brakeman.	Set up automatic checks on every change. We do it so two parallel tracks catch breakage early instead of drifting apart.
Phase 0: Foundation (Sprint 0)	F-06	Data model	• Migrations and models for restaurant_groups, locations, and location_config (tunables as rows).	Build the tables for groups, locations, and per location settings. We do it because every record hangs off a location, and the settings table holds tunable numbers instead of hard coding them.
Phase 0: Foundation (Sprint 0)	F-07	Data model	• Migrations and models for staff (persistent identity) and shifts (per night state, rotation spot, cut status, covers).	Build the server identity table and the separate per night shift table. We split them because a server's skill carries across nights but their rotation spot resets nightly.

Sprint 0: Shared Foundation (split by component)

Goal: Stand up the Rails 8 monolith, schema, realtime plumbing, auth and multi tenancy, core domain models, and the synthetic event source. Both devs are blocked on rr system and own models in parallel, then begins ROI Internal location setup (I-01) the moment the config models land.

Phase / Sprint	Task ID	Epic	Task (bullet)	What it means + why we do it
Phase 0: Foundation (Sprint 0)	F-08	Data model	• Migrations and models for sections and tables (status, seated_at, projected_free_at, combinable, size cap).	Build the floor tables and the sections that group them. We do it because the engine and the floor plan both read this to decide who can seat a party.
Phase 0: Foundation (Sprint 0)	F-09	Data model	• Migrations and models for guests, parties (VIP, HNWL, recovery flags) and reservations (hold table, hold release).	Build the guest, party, and reservation tables with valued guest flags and table hold fields. We do it so the engine can later route valued guests and hold their table.
Phase 0: Foundation (Sprint 0)	F-10	Data model	• Migrations for raw_events as a RANGE partitioned, append only table (monthly partitions) with an idempotency index.	Build the events table that records every table lifecycle event, split by month, never edited or deleted. We do it because this is the permanent source of truth the engine runs on and ML later trains on.
Phase 0: Foundation (Sprint 0)	F-11	Data model	• Migrations for seating_decisions (pool, eliminations, scores, host action, engine flag) and decision_outcomes.	Build the log that records, on every decision, the candidate pool, what got filtered, the scores, and what the host did. We do it because even with no ML today this becomes labelled training data for Phase 2, for free.
Phase 0: Foundation (Sprint 0)	F-12	Auth & tenancy	• Authentication plus three roles: head host, greeter, manager. Session based for web.	Add login and the role system. We do it because every screen and permission depends on who is signed in, for example only a manager can approve a cut.
Phase 0: Foundation (Sprint 0)	F-13	Auth & tenancy	• Location scoping on all tenant tables plus a current location resolver.	Make sure every query is limited to one location so data never leaks between restaurants. We do it because a multi location product cannot show one location's floor to another.
Phase 0: Foundation (Sprint 0)	F-14	Real-time	• ActionCable and Turbo Streams on Solid Cable (Postgres backed), a per location floor state channel, plus the floor view binding glue.	Set up the live push system so a change on one screen instantly appears on all others, and the wiring the Host floor view hooks into. We do it because the whole product depends on every screen sharing one live picture of the floor.

Sprint 0: Shared Foundation (split by component)

Goal: Stand up the Rails 8 monolith, schema, realtime plumbing, auth and multi tenancy, core domain models, and the synthetic event source. Both devs are blocked on rr system and own models in parallel, then begins ROI Internal location setup (I-01) the moment the config models land.

Phase / Sprint	Task ID	Epic	Task (bullet)	What it means + why we do it
Phase 0: Foundation (Sprint 0)	F-15	Real-time	• Solid Queue background jobs on Postgres, with one example job running.	Set up background job processing. We do it because pacing and cut checks run in the background later, and this avoids needing Redis.
Phase 0: Foundation (Sprint 0)	F-16	POS harness	• Synthetic event generator that fires a realistic stream of table lifecycle events at a chosen pace.	Build a tool that fakes a realistic service night. We do it so both devs can build and test the engine and live floor without waiting on the real POS, which is still unconfirmed.
Phase 0: Foundation (Sprint 0)	F-17	POS harness	• Ingestion adapter interface and webhook endpoint stub. The generator and the future Squirrel feed both go through it.	Build the single doorway all events come through, fake now or real later. We do it so swapping to the real POS is a one piece change, not an engine rewrite.
Phase 0: Foundation (Sprint 0)	F-18	Design system	• Shared Hotwire layout, Stimulus setup, base styles and reusable component partials used across all surfaces.	Build the shared layout and reusable UI pieces. We do it to stop the two of you inventing two different looking UIs.
Phase 0: Foundation (Sprint 0)	I-01	ROI Internal	• Location setup screen: create a location, set timezone, POS provider, reservation source, and edit the tunables.	A screen for the ROI team to create a location and edit its settings. Pulled into Sprint 0 so Kedi has independent work as soon as the config models (F-06) and tenancy (F-13) exist, instead of waiting on the rest of foundation.

Sprint 0: Shared Foundation (split k

Goal: Stand up the Rails 8 monolith, schema, realtimost of this, so Tanner does the CRITICAL PATH foundation first to unblock Kedi fast. Kedi works the design system and own models in parallel, then begins ROI

Phase / Sprint	Task ID	Epic	Owner	Depends on	Est (days)	Priority	Status	Done when
Phase 0: Foundation (Sprint 0)	F-01	Repo & tooling	Tanner	none	1	CRITICAL PATH	Not started	App boots, Turbo and Stimulus present by default.
Phase 0: Foundation (Sprint 0)	F-02	Repo & tooling	Tanner	F-01	0.5	CRITICAL PATH	Not started	schema_format = :sql committed and verified.
Phase 0: Foundation (Sprint 0)	F-03	Local DB env	Kedi	F-01	1	High	Not started	A new dev runs either path and db:setup works.
Phase 0: Foundation (Sprint 0)	F-04	Local DB env	Kedi	F-03	1	High	Not started	db:seed creates a location, roster and floor.
Phase 0: Foundation (Sprint 0)	F-05	CI / quality	Kedi	F-01	1	Medium	Not started	PRs show green checks before merge.
Phase 0: Foundation (Sprint 0)	F-06	Data model	Kedi	F-02	1	CRITICAL PATH	Not started	location_config holds idle minutes, fairness band, skill thresholds, hold minutes, large party threshold.
Phase 0: Foundation (Sprint 0)	F-07	Data model	Tanner	F-02	1.5	CRITICAL PATH	Not started	staff and shifts split intact with associations.

Sprint 0: Shared Foundation (split k

Goal: Stand up the Rails 8 monolith, schema, realtime most of this, so Tanner does the CRITICAL PATH foundation first to unblock Kedi fast. Kedi works the design system and own models in parallel, then begins ROI

Phase / Sprint	Task ID	Epic	Owner	Depends on	Est (days)	Priority	Status	Done when
Phase 0: Foundation (Sprint 0)	F-08	Data model	Tanner	F-02	1.5	CRITICAL PATH	Not started	Floor entities location scoped, combinable and size cap present.
Phase 0: Foundation (Sprint 0)	F-09	Data model	Kedi	F-02	1.5	High	Not started	Reservation hold fields present for VIP hold logic.
Phase 0: Foundation (Sprint 0)	F-10	Data model	Tanner	F-02,F-07,F-08	2	CRITICAL PATH	Not started	Partition DDL in a migration, full raw payload in jsonb.
Phase 0: Foundation (Sprint 0)	F-11	Data model	Tanner	F-02,F-07	1.5	High	Not started	A decision row is written on every engine run.
Phase 0: Foundation (Sprint 0)	F-12	Auth & tenancy	Kedi	F-06	1.5	CRITICAL PATH	Not started	Login works for each role, greeter restricted versus head host.
Phase 0: Foundation (Sprint 0)	F-13	Auth & tenancy	Kedi	F-06,F-12	1.5	High	Not started	No query leaks across locations.
Phase 0: Foundation (Sprint 0)	F-14	Real-time	Tanner	F-01	1.75	CRITICAL PATH	Not started	A change on one screen appears on another in under a second locally.

Sprint 0: Shared Foundation (split k

Goal: Stand up the Rails 8 monolith, schema, realtime most of this, so Tanner does the CRITICAL PATH foundation first to unblock Kedi fast. Kedi works the design system and own models in parallel, then begins ROI

Phase / Sprint	Task ID	Epic	Owner	Depends on	Est (days)	Priority	Status	Done when
Phase 0: Foundation (Sprint 0)	F-15	Real-time	Tanner	F-01	0.5	Medium	Not started	A sample background job runs on the queue.
Phase 0: Foundation (Sprint 0)	F-16	POS harness	Tanner	F-10	2.5	CRITICAL PATH	Not started	Can simulate a full service night of events.
Phase 0: Foundation (Sprint 0)	F-17	POS harness	Tanner	F-16	1.5	High	Not started	Swapping synthetic to real POS is one adapter change.
Phase 0: Foundation (Sprint 0)	F-18	Design system	Kedi	F-01	2	High	Not started	One visual system reused by Host, Manager and Internal.
Phase 0: Foundation (Sprint 0)	I-01	ROI Internal	Kedi	F-06,F-13	2	High	Not started	A new location can be stood up end to end.

Sprint 1: ROI Internal (onboarding) plus Deterministic Engine core

Goal: Kedi builds ROI Internal so a location can be configured, including the drag and drop floor plan (now split into two parts). Tanner builds the deterministic rule engine,

Phase / Sprint	Task ID	Epic	Task (bullet)	What it means + why we do it
Phase 1: Onboarding + Engine Core (Sprint 1)	I-02	ROI Internal	• Roster setup screen: add staff, set role, skill score and section, in a shape ready for a future Dayforce import.	A screen to add staff and set role, skill score and section. We do it because the rotation and the engine's strong or weak routing both need a real roster with skill scores.
Phase 1: Onboarding + Engine Core (Sprint 1)	I-03	ROI Internal	• Floor plan editor part 1: the drag and drop canvas, placing tables, setting seat counts and table types (Stimulus + SVG).	The first half of the visual floor editor: the canvas and placing tables. We split this big build in two so it is easier to manage and review. This is the map the engine and live floor read.
Phase 1: Onboarding + Engine Core (Sprint 1)	I-03b	ROI Internal	• Floor plan editor part 2: grouping tables into sections and marking which tables are combinable.	The second half of the floor editor: sections and combinable marking. We split it from part 1 so each piece is a clean unit of work.
Phase 1: Onboarding + Engine Core (Sprint 1)	I-04	ROI Internal	• Section size cap derivation from combinable tables, plus validation that no table is orphaned.	Work out section capacity from combinable tables and check no table is left unassigned. We do it because bad floor data would make the engine try to seat parties where they cannot fit.
Phase 1: Onboarding + Engine Core (Sprint 1)	E-01	Engine - rules	• Rule engine scaffold: an ordered pipeline that takes a party plus live floor and returns the candidate pool through staged filters, logging each stage.	The ordered pipeline that runs an arriving party through the staged filters and logs each stage. We do it because this is the backbone of the seating decision and the must never be wrong core.
Phase 1: Onboarding + Engine Core (Sprint 1)	E-02	Engine - rules	• Hard constraint: shift gate (exclude not yet started servers) plus cut protection (exclude flagged servers).	Filter out servers who are not on shift yet or who are flagged to go home. We do it because you never seat a server who has not started or is leaving.
Phase 1: Onboarding + Engine Core (Sprint 1)	E-03	Engine - rules	• Hard constraint: section fit, including combinable table logic for large parties.	Filter out sections too small for the party, using combinable table logic for big groups. We do it because a party of eight cannot go to a two top.

Sprint 1: ROI Internal (onboarding) plus Deterministic Engine core

Goal: Kedi builds ROI Internal so a location can be configured, including the drag and drop floor plan (now split into two parts). Tanner builds the deterministic rule engine,

Phase / Sprint	Task ID	Epic	Task (bullet)	What it means + why we do it
Phase 1: Onboarding + Engine Core (Sprint 1)	E-04	Engine - rules	• Hard constraint: large party restriction plus VIP, HNW and recovery pre filter (restrict to strong servers).	Filter weak and new servers out of large parties, and restrict valued guests to strong servers. We do it as risk management. Blocked until the skill score formula (D-01) is decided.
Phase 1: Onboarding + Engine Core (Sprint 1)	E-05	Engine - rules	• Rotation pointer: order by shift start, handle new server insertion, and wrap around.	Order servers by start time, handle a new server joining the back of the line, and wrap around. We do it because rotation is the fair queue the whole engine is built on.
Phase 1: Onboarding + Engine Core (Sprint 1)	E-06	Engine - tests	• Deterministic engine test harness: scripted synthetic nights asserting pool and selection correctness, with the tiebreak.	A test harness of scripted nights that proves the engine always gives the same explainable answer. We do it because the core must be provably correct before it ever runs a real floor.

Sprint 1: ROI Internal (onboarding)

Goal: Kedi builds ROI Internal so a location can be α the hard constraints that must never be wrong.

Phase / Sprint	Task ID	Epic	Owner	Depends on	Est (days)	Priority	Status	Done when
Phase 1: Onboarding + Engine Core (Sprint 1)	I-02	ROI Internal	Kedi	F-07	2	High	Not started	A full roster can be entered, skill score editable.
Phase 1: Onboarding + Engine Core (Sprint 1)	I-03	ROI Internal	Kedi	F-08,F-18	2	High	Not started	Tables can be placed with seats and types on a saved canvas.
Phase 1: Onboarding + Engine Core (Sprint 1)	I-03b	ROI Internal	Kedi	I-03	2	High	Not started	Tables group into sections, combinable tables are flagged.
Phase 1: Onboarding + Engine Core (Sprint 1)	I-04	ROI Internal	Kedi	I-03b	1.5	Medium	Not started	Every table belongs to a section, combinable logic gives the right cap.
Phase 1: Onboarding + Engine Core (Sprint 1)	E-01	Engine - rules	Tanner	F-07,F-08,F-10,F-11	2	CRITICAL PATH	Not started	Pipeline runs end to end on synthetic events, each stage recorded.
Phase 1: Onboarding + Engine Core (Sprint 1)	E-02	Engine - rules	Tanner	E-01	1	High	Not started	Off shift or flagged servers never appear in the pool.
Phase 1: Onboarding + Engine Core (Sprint 1)	E-03	Engine - rules	Tanner	E-01,I-03b	2	High	Not started	A party of N only routes to sections that physically fit.

Sprint 1: ROI Internal (onboarding)

Goal: Kedi builds ROI Internal so a location can be ccthe hard constraints that must never be wrong.

Phase / Sprint	Task ID	Epic	Owner	Depends on	Est (days)	Priority	Status	Done when
Phase 1: Onboarding + Engine Core (Sprint 1)	E-04	Engine - rules	Tanner	E-01,D-01	1.5	High	Not started	Valued and large parties only route to strong servers.
Phase 1: Onboarding + Engine Core (Sprint 1)	E-05	Engine - rules	Tanner	E-01,D-05	1.5	High	Not started	New server behaviour matches the D-05 decision.
Phase 1: Onboarding + Engine Core (Sprint 1)	E-06	Engine - tests	Tanner	E-02,E-03,E-04,E-05	2	High	Not started	Same input always yields the same, explainable output.

Sprint 2: Floor views (Kedi) plus scoring, cut-rec and VIP holds (Tanner)

Goal: Kedi builds the Host floor and seating views and the Manager live view, all reading the same realtime state. Tanner adds heuristic scoring and, to stay busy in parallel forward. First real integration of the two tracks, so settle the X-01 channel contract first.

Phase / Sprint	Task ID	Epic	Task (bullet)	What it means + why we do it
Phase 2: Scoring + Floor Views (Sprint 2)	E-07	Engine - scoring	• Scoring interface: each soft factor is a swappable strategy (heuristic now, Python ML call later) that ranks survivors of the rule pool.	Build the ranking step as swappable parts, heuristic today and an ML call later. We do it because this clean seam is exactly where Phase 2 ML plugs in with no rewrite.
Phase 2: Scoring + Floor Views (Sprint 2)	E-08	Engine - scoring	• Heuristic fairness score (covers vs floor average, penalty only beyond the band) plus availability score (idle minutes boost).	Heuristic scores that gently penalise a server far ahead of average and boost one sitting idle. We do it to keep rotation fair without punishing normal variation.
Phase 2: Scoring + Floor Views (Sprint 2)	E-09	Engine - scoring	• Heuristic projected availability (fixed turn time by party size) plus efficiency score (covers per hour vs personal baseline).	Heuristic guesses of when a server's table frees, plus a reward for beating their own pace. We do it because covers per hour, not raw headcount, is the true measure so a fast server is never skipped.
Phase 2: Scoring + Floor Views (Sprint 2)	H-01	ROI Host	• Render the saved floor plan as a live SVG on the Host screen.	Draw the host's floor on screen from the saved plan. We do it because the host needs to see the real floor to run it. (Frontend work, moved to Kedi in the rebalance.)
Phase 2: Scoring + Floor Views (Sprint 2)	H-01b	ROI Host	• Bind the floor view to the live channel so table status updates appear in real time.	Wire the floor view to the live channel so seated, dirty, available and held update instantly. We split this from H-01 so the rendering and the live wiring are clean units.
Phase 2: Scoring + Floor Views (Sprint 2)	H-02	ROI Host	• Greeter check in and walk in flow: the form, party creation, and waitlist entry.	The greeter flow to check guests in and add walk ins. We do it because walk ins must enter the system, and the greeter is limited so they cannot bypass seating control. (Moved to Kedi.)
Phase 2: Scoring + Floor Views (Sprint 2)	H-03	ROI Host	• Seating recommendation card UI: show the engine's suggested table and server on the Host screen.	Show the engine's suggestion on the host screen. We do it because this is the visible half of the core loop. The suggestion itself is produced by Tanner's engine (E-07). (Moved to Kedi.)

Sprint 2: Floor views (Kedi) plus scoring, cut-rec and VIP holds (Tanner)

Goal: Kedi builds the Host floor and seating views and the Manager live view, all reading the same realtime state. Tanner adds heuristic scoring and, to stay busy in parallel forward. First real integration of the two tracks, so settle the X-01 channel contract first.

Phase / Sprint	Task ID	Epic	Task (bullet)	What it means + why we do it
Phase 2: Scoring + Floor Views (Sprint 2)	H-03b	ROI Host	• Confirm and override controls plus the override reason modal, recording the host action to the decision log.	The buttons the host uses to accept or override the suggestion, and the reason capture. We split this from H-03 so the display and the interaction are separate units. The logging teaches Phase 2.
Phase 2: Scoring + Floor Views (Sprint 2)	M-01	ROI Manager	• Manager live floor status: a read only realtime view of covers, table states and per server load.	The manager's read only live view of the floor. We do it to give the GM the realtime picture the product promises.
Phase 2: Scoring + Floor Views (Sprint 2)	M-02	ROI Manager	• Live server load and covers per hour panel per shift.	A live panel of each server's load and covers per hour. We do it to show the manager the same fairness signals the engine uses.
Phase 2: Scoring + Floor Views (Sprint 2)	C-01	Cuts	• Cut recommendation service: decide when and how many can safely go home in strict start order, guarding against a forecast late rush.	Work out when and how many servers can go home, in start order, without cutting before a likely late rush. Pulled into Sprint 2 so Tanner has engine backend work here while Kedi builds the floor views, since its only prerequisite (E-05) is done in Sprint 1.
Phase 2: Scoring + Floor Views (Sprint 2)	V-01	Special guests	• Reservation table hold: hold a good table for a valued reservation, protect it from walk ins until 15 minutes past booking, then release.	Hold a good table for a valued reservation and protect it until fifteen minutes past the booking, then release. Pulled into Sprint 2 alongside C-01 to balance Tanner's load, its prerequisites (E-04, F-09) are already done.
Phase 2: Scoring + Floor Views (Sprint 2)	X-01	Integration	• SHARED SEAM: agree in writing the Host and Manager floor state channel contract (event names, payload shape).	Both devs agree in writing the exact shape of the live floor updates before building. We do it because this prevents the two halves breaking each other during integration.

Sprint 2: Floor views (Kedi) plus sc

Goal: Kedi builds the Host floor and seating views and rather than idle while the views are built, pulls the cut recommendation (C-01) and VIP hold (V-01) logic forward. First real integration of the two tracks, so set

Phase / Sprint	Task ID	Epic	Owner	Depends on	Est (days)	Priority	Status	Done when
Phase 2: Scoring + Floor Views (Sprint 2)	E-07	Engine - scoring	Tanner	E-06	1.5	High	Not started	Interface signature stable, today returns heuristics.
Phase 2: Scoring + Floor Views (Sprint 2)	E-08	Engine - scoring	Tanner	E-07,D-02,D-03	1.5	High	Not started	Reads fairness band and idle threshold from config.
Phase 2: Scoring + Floor Views (Sprint 2)	E-09	Engine - scoring	Tanner	E-07	1.5	High	Not started	Cold start defaults in place, efficiency rewards speed.
Phase 2: Scoring + Floor Views (Sprint 2)	H-01	ROI Host	Kedi	F-14,I-03	1.25	High	Not started	The saved plan renders on the Host screen.
Phase 2: Scoring + Floor Views (Sprint 2)	H-01b	ROI Host	Kedi	H-01,F-14	1	High	Not started	Table status changes appear live in under a second.
Phase 2: Scoring + Floor Views (Sprint 2)	H-02	ROI Host	Kedi	F-09,F-12	1.5	Medium	Not started	Greeter can add walk ins, cannot seat.
Phase 2: Scoring + Floor Views (Sprint 2)	H-03	ROI Host	Kedi	E-07,H-01	1.25	High	Not started	The recommended table and server display clearly.

Sprint 2: Floor views (Kedi) plus scoring

Goal: Kedi builds the Host floor and seating views and rather than idle while the views are built, pulls the cut recommendation (C-01) and VIP hold (V-01) logic forward. First real integration of the two tracks, so set

Phase / Sprint	Task ID	Epic	Owner	Depends on	Est (days)	Priority	Status	Done when
Phase 2: Scoring + Floor Views (Sprint 2)	H-03b	ROI Host	Kedi	H-03	1.25	High	Not started	Confirm and override both recorded to the decision log.
Phase 2: Scoring + Floor Views (Sprint 2)	M-01	ROI Manager	Kedi	F-14,H-01	2.5	High	Not started	Manager sees the same floor as the host, plus aggregates.
Phase 2: Scoring + Floor Views (Sprint 2)	M-02	ROI Manager	Kedi	F-07,E-09	1.5	Medium	Not started	Per server load and covers per hour visible live.
Phase 2: Scoring + Floor Views (Sprint 2)	C-01	Cuts	Tanner	E-05	2	High	Not started	Recommends readiness by start order, every recommendation logged.
Phase 2: Scoring + Floor Views (Sprint 2)	V-01	Special guests	Tanner	E-04,F-09	2	High	Not started	Hold and auto release at the configured minutes.
Phase 2: Scoring + Floor Views (Sprint 2)	X-01	Integration	Kedi + Tanner	F-14	1	CRITICAL PATH	Not started	A written contract both build against exists.

Sprint 3: Pacing, cuts and rush (Tanner) plus analytics begins (Kedi)

Goal: Tanner builds the pacing governor, cut execution and rush enforcement. With the cut recommendation and VIP hold already done in Sprint 2, his lane here is lighter, for Sprint 4. The cut approval flow is a shared seam. Most integration risk lives here.

Phase / Sprint	Task ID	Epic	Task (bullet)	What it means + why we do it
Phase 3: Pacing, Cuts, Rush (Sprint 3)	P-01	Pacing	• Pacing governor: compute the floor load index and kitchen load index from the event stream.	Compute live numbers for how stressed the floor and kitchen are. We do it because these drive the decision to slow seating before service collapses.
Phase 3: Pacing, Cuts, Rush (Sprint 3)	P-02	Pacing	• Hold recommendation: when an index crosses its threshold, suggest a hold the host accepts or declines. Respects the rush flag. Never auto holds.	When stress is high, suggest holding the next seat, which the host can accept or decline, and stand down while rush is on. We do it because pacing protects service but the host stays in control.
Phase 3: Pacing, Cuts, Rush (Sprint 3)	C-02	Cuts	• SHARED SEAM: cut approval flow. Host flags ready, manager approves with a code or device tap, then the engine executes.	Host flags ready to cut, manager approves, then the engine acts. We do it jointly because this spans both surfaces, only a manager can approve, and the contract must be shared.
Phase 3: Pacing, Cuts, Rush (Sprint 3)	C-03	Cuts	• Cut execution: the cut server stops getting tables, finishes their own, and cleared tables route to the nearest active server as pickups.	A cut server stops getting tables, finishes their own, and their cleared tables pass to the nearest active server. We do it so nobody runs the room, and the closer advantage is preserved.
Phase 3: Pacing, Cuts, Rush (Sprint 3)	C-04	Cuts	• Cut cost visibility: a manager view of recommendations versus approvals and time kept on past the recommendation.	A manager view of cut recommendations versus approvals and time kept on past them. We do it so the business can see money spent keeping staff longer than needed.
Phase 3: Pacing, Cuts, Rush (Sprint 3)	R-01	Rush mode	• Rush mode toggle and indicator: sets the rush flag and tags seats made during the rush. (Suspension itself is enforced by P-02 and D-SEAT reading this flag.)	The host's rush button and active indicator, which sets the flag and tags rushed seats. We do it so the host can load the floor fast in a surge. The actual standing down of holds happens in Tanner's pacing tasks. (Frontend, moved to Kedi.)
Phase 3: Pacing, Cuts, Rush (Sprint 3)	D-SEAT	Engine - rules	• Double seat protection within a short window, suspended while the rush flag is on.	Stop two parties going to one server at almost the same moment, except during a rush. We do it to prevent accidentally burying a server.

Sprint 3: Pacing, cuts and rush (Tanner) plus analytics begins (Kedi)

Goal: Tanner builds the pacing governor, cut execution and rush enforcement. With the cut recommendation and VIP hold already done in Sprint 2, his lane here is lighter, for Sprint 4. The cut approval flow is a shared seam. Most integration risk lives here.

Phase / Sprint	Task ID	Epic	Task (bullet)	What it means + why we do it
Phase 3: Pacing, Cuts, Rush (Sprint 3)	M-03	ROI Manager	• No show tracking: per guest no show history, reservation status surfacing, manager review.	Track no shows per guest and surface reservation status for review. Pulled into Sprint 3 so Kedi has analytics work here while Tanner finishes pacing and cuts, its only prerequisite (F-09) is long done.
Phase 3: Pacing, Cuts, Rush (Sprint 3)	M-04	ROI Manager	• Server and guest analytics for the location: covers, pacing history, per server performance over a shift or period.	A manager dashboard of covers, pacing history, and per server performance. Pulled into Sprint 3 to balance Kedi's lane. Order it after P-01 within the sprint, since it reads the pacing history P-01 produces.

Sprint 3: Pacing, cuts and rush (Tar

Goal: Tanner builds the pacing governor, cut executico Kedi starts the analytics work (no show tracking and the analytics dashboard) in parallel rather than waiting for Sprint 4. The cut approval flow is a shared seam.

Phase / Sprint	Task ID	Epic	Owner	Depends on	Est (days)	Priority	Status	Done when
Phase 3: Pacing, Cuts, Rush (Sprint 3)	P-01	Pacing	Tanner	F-16,E-09	2	High	Not started	Indices computed live as a background job.
Phase 3: Pacing, Cuts, Rush (Sprint 3)	P-02	Pacing	Tanner	P-01,H-03,R-01	1.5	High	Not started	Recommendation only, suspended during rush, logged.
Phase 3: Pacing, Cuts, Rush (Sprint 3)	C-02	Cuts	Kedi + Tanner	C-01,M-01	2	CRITICAL PATH	Not started	Head host cannot self approve, pop up hits host and manager at once.
Phase 3: Pacing, Cuts, Rush (Sprint 3)	C-03	Cuts	Tanner	C-02	2	High	Not started	Pickup re routing works on a synthetic night.
Phase 3: Pacing, Cuts, Rush (Sprint 3)	C-04	Cuts	Kedi	C-01	1.5	Medium	Not started	Manager can see spend on labour kept beyond recommended cut.
Phase 3: Pacing, Cuts, Rush (Sprint 3)	R-01	Rush mode	Kedi	D-04	1.5	High	Not started	Toggle sets the flag, rushed seats are tagged out of baselines.
Phase 3: Pacing, Cuts, Rush (Sprint 3)	D-SEAT	Engine - rules	Tanner	E-07,R-01	1	Medium	Not started	Two near simultaneous parties to one server are prevented outside rush.

Sprint 3: Pacing, cuts and rush (Tar

Goal: Tanner builds the pacing governor, cut executicso Kedi starts the analytics work (no show tracking and the analytics dashboard) in parallel rather than waiting for Sprint 4. The cut approval flow is a shared seam. .

Phase / Sprint	Task ID	Epic	Owner	Depends on	Est (days)	Priority	Status	Done when
Phase 3: Pacing, Cuts, Rush (Sprint 3)	M-03	ROI Manager	Kedi	F-09	1.5	Medium	Not started	No show counts accrue per guest.
Phase 3: Pacing, Cuts, Rush (Sprint 3)	M-04	ROI Manager	Kedi	M-02,P-01	2.5	Medium	Not started	Dashboard reads from the decision and pacing logs.

Sprint 4: Finish analytics, decision audit view, hardening, shadow mode pilot readiness

Goal: Kedi finishes the analytics (the U shape diagnostic) and the decision audit view. Both devs harden, then prepare for a shadow mode pilot: ROI recommends, the hun onboarding.

Phase / Sprint	Task ID	Epic	Task (bullet)	What it means + why we do it
Phase 4: Analytics + Pilot (Sprint 4)	M-05	ROI Manager	• U shape diagnostic: a nightly covers by start order curve that flags when it goes flat.	A nightly curve of covers by start order that should look like a U, flagging if it goes flat. We do it because a flat curve means the engine is wrongly equalising instead of protecting openers and closers.
Phase 4: Analytics + Pilot (Sprint 4)	Q-01	Hardening	• Decision log audit view (internal): replay any night's decisions step by step, pool to eliminations to score to selection to host action.	An internal view to replay any night's decisions step by step, reading the log the engine writes. We do it to prove the engine is sound during shadow mode. (Frontend view, moved to Kedi.)
Phase 4: Analytics + Pilot (Sprint 4)	Q-03	Hardening	• Offline and disconnect resilience for Host: a kiosk wifi blip must not stop seating, with reconnect and state re sync.	Make the host screen survive a wifi blip and resync. We do it because a kiosk dropping connection cannot be allowed to stop seating.
Phase 4: Analytics + Pilot (Sprint 4)	Q-02	Hardening	• SHARED: performance pass. Realtime fanout under a full simulated service, an N+1 sweep, a Solid Cable throughput check, decide if Redis is needed.	Stress test the live updates and queries under a full simulated night and decide if Redis is needed. We do it because the floor must stay responsive at peak.
Phase 4: Analytics + Pilot (Sprint 4)	Q-04	Hardening	• SHARED: end to end synthetic night acceptance test, a full service from open to close with all features firing.	One scripted full night that exercises every feature and asserts the U shape and auditability. We do it as a single green run that proves the whole system works end to end.
Phase 4: Analytics + Pilot (Sprint 4)	PILOT-01	Pilot prep	• Shadow mode runbook plus onboarding one real pilot location via ROI Internal, with OpenTable read or manual entry.	Use ROI Internal to fully set up one real pilot location and wire its reservations. We do it because a configured real location is the prerequisite for any pilot.
Phase 4: Analytics + Pilot (Sprint 4)	PILOT-02	Pilot prep	• Squirrel or Omnivore confirmation plus the real webhook behind the adapter if available, else continue shadow on synthetic.	Confirm the POS can emit the feed and drop the real webhook in behind the adapter, or continue in shadow mode without it. We do it because real data improves the pilot, but the adapter means we are not blocked.

Sprint 4: Finish analytics, decision

Goal: Kedi finishes the analytics (the U shape diagonal host executes, everything logged. This is the lightest sprint by design, a buffer for hardening and pilot onboarding.

Phase / Sprint	Task ID	Epic	Owner	Depends on	Est (days)	Priority	Status	Done when
Phase 4: Analytics + Pilot (Sprint 4)	M-05	ROI Manager	Kedi	M-04	1.5	Medium	Not started	The curve renders per night and flags a flat shape.
Phase 4: Analytics + Pilot (Sprint 4)	Q-01	Hardening	Kedi	E-06,H-03	2	High	Not started	Any night can be replayed and explained.
Phase 4: Analytics + Pilot (Sprint 4)	Q-03	Hardening	Tanner	H-01b	2	High	Not started	Host recovers cleanly from a dropped connection mid service.
Phase 4: Analytics + Pilot (Sprint 4)	Q-02	Hardening	Kedi + Tanne	H-01b,M-01	2	High	Not started	Floor stays responsive under peak synthetic load.
Phase 4: Analytics + Pilot (Sprint 4)	Q-04	Hardening	Kedi + Tanne	Q-01	2	High	Not started	One scripted night exercises the whole system green.
Phase 4: Analytics + Pilot (Sprint 4)	PILOT-01	Pilot prep	Kedi	I-01,I-02,I-03b	2	High	Not started	One real location fully configured.
Phase 4: Analytics + Pilot (Sprint 4)	PILOT-02	Pilot prep	Tanner	F-17,DEP-01	1.5	High	Not started	Real POS drops in behind the adapter, or pilot proceeds without it.

Open Decisions: resolve in parallel (most block Tanner's engine)

These are open items from the spec. Owner is Kedi (product judgement) unless noted. Each lists the task it blocks.

ID	Decision needed	Why it blocks (plain)	Blocks	Owner	Status
D-01	The single skill score formula (strong is 70 or more, weak is under 40) used the same way for section sizing, large party restriction and the VIP pre filter.	The engine cannot run the large party restriction or VIP routing without a concrete strong or weak number to compare against.	E-04	Kedi	Open
D-02	Fairness band percent (spec suggests 15) for how far ahead a server can get before a penalty applies.	Sets when the fairness score starts re ranking. It lives in config but still needs a starting value.	E-08	Kedi	Open
D-03	Availability idle threshold (spec says 8 minutes) before an idle server is boosted.	Drives the availability score. It needs a confirmed default to compute against.	E-08	Kedi	Open
D-04	Rush mode scope: floor wide or targeted at one server.	Changes how the rush toggle is built and how rush tagging is scoped in R-01.	R-01	Kedi	Open
D-05	New server rotation insertion: confirm a newly started server takes the very next table, then resumes the normal cycle.	Determines the rotation pointer behaviour in E-05.	E-05	Kedi	Open
D-06	Strong server erosion guardrail: keep at least one combinable strong section until last seating.	Affects the cut logic, there is a risk of no strong section left for a late VIP or large party.	C-01,C-03	Kedi + Tanner	Open
DEP-01	Squirrel or Omnivore: can it emit the table lifecycle stream, and do food_delivered and table_available exist in it.	Critical path for real data. MVP1 runs on synthetic, but pilot data quality depends on this answer.	PILOT-02	Kedi	Open
D-07	Dayforce read: confirm how the roster and start times are pulled. Manual entry is acceptable for MVP1.	Affects the roster import shape in I-02. Manual entry is the MVP1 fallback.	I-02	Kedi	Open

Workload Split - exact 50/50 check

Owned tasks are split 27/27. Shared tasks are done jointly (counted half to each). With shared added in, each dev carries 46.25 days

Phase / Sprint	Tanner tasks	Tanner days	Kedi tasks	Kedi days	Shared tasks	Shared days
Phase 0: Foundation (Sprint 0)	10	14.25	9	12.5	0	0
Phase 1: Onboarding + Engine Core (Sprint 1)	6	10	4	7.5	0	0
Phase 2: Scoring + Floor Views (Sprint 2)	5	8.5	7	10.25	1	1
Phase 3: Pacing, Cuts, Rush (Sprint 3)	4	6.5	4	7	1	2
Phase 4: Analytics + Pilot (Sprint 4)	2	3.5	3	5.5	2	4
TOTAL (owned)	27	42.75	27	42.75	4	7

With shared split evenly (half each):

Dev	Task equiv	Days	Share
Tanner	29	46.25	50.0%
Kedi	29	46.25	50.0%

Parallel Flow - the two lanes, sprint by sprint

Each sprint runs as two concurrent lanes. The per sprint day load is now balanced (worst gap 2.5 days) so neither dev idles while

Sprint (lane load)	Tanner lane	Kedi lane
Phase 0: Foundation (Sprint 0) Tanner 14.25d / Kedi 12.5d lane gap +1.75d	F-01 Initialize Rails 8.1.x app (1.0d) F-02 Set schema format to (0.5d) F-07 Migrations and models for staff (1.5d) F-08 Migrations and models for sections and tables (1.5d) F-10 Migrations for raw_events as a RANGE partiti.. (2.0d) F-11 Migrations for seating_decisions (1.5d) F-14 ActionCable and Turbo Streams on Solid Cable (1.75d) F-15 Solid Queue background jobs on Postgres, wit.. (0.5d) F-16 Synthetic event generator that fires a reali.. (2.5d) F-17 Ingestion adapter interface and webhook endp.. (1.5d)	F-03 docker-compose.yml with a Postgres 15 servic.. (1.0d) F-04 Migration workflow doc plus a seed script th.. (1.0d) F-05 GitHub Actions running migrations and tests .. (1.0d) F-06 Migrations and models for restaurant_groups,.. (1.0d) F-09 Migrations and models for guests, parties (1.5 d) F-12 Authentication plus three roles (1.5d) F-13 Location scoping on all tenant tables plus a.. (1.5d) F-18 Shared Hotwire layout, Stimulus setup, base .. (2.0d) I-01 Location setup screen (2.0d)
Phase 1: Onboarding + Engine Core (Sprint 1) Tanner 10.0d / Kedi 7.5d lane gap +2.50d	E-01 Rule engine scaffold (2.0d) E-02 Hard constraint (1.0d) E-03 Hard constraint (2.0d) E-04 Hard constraint (1.5d) E-05 Rotation pointer (1.5d) E-06 Deterministic engine test harness (2.0d)	I-02 Roster setup screen (2.0d) I-03 Floor plan editor part 1 (2.0d) I-03b Floor plan editor part 2 (2.0d) I-04 Section size cap derivation from combinable .. (1.5d)

Phase 2: Scoring + Floor Views (Sprint 2) Tanner 8.5d / Kedi 10.25d lane gap -1.75d	E-07 Scoring interface (1.5d) E-08 Heuristic fairness score (1.5d) E-09 Heuristic projected availability (1.5d) C-01 Cut recommendation service (2.0d) V-01 Reservation table hold (2.0d)	H-01 Render the saved floor plan as a live SVG on.. (1.25d) H-01b Bind the floor view to the live channel so t.. (1.0d) H-02 Greeter check in and walk in flow (1.5d) H-03 Seating recommendation card UI (1.25d) H-03b Confirm and override controls plus the overr.. (1.25d) M-01 Manager live floor status (2.5d) M-02 Live server load and covers per hour panel p.. (1.5d)
Phase 3: Pacing, Cuts, Rush (Sprint 3) Tanner 6.5d / Kedi 7.0d lane gap -0.50d	P-01 Pacing governor (2.0d) P-02 Hold recommendation (1.5d) C-03 Cut execution (2.0d) D-SEAT Double seat protection within a short window.. (1.0d)	C-04 Cut cost visibility (1.5d) R-01 Rush mode toggle and indicator (1.5d) M-03 No show tracking (1.5d) M-04 Server and guest analytics for the location (2.5d)
Phase 4: Analytics + Pilot (Sprint 4) Tanner 3.5d / Kedi 5.5d lane gap -2.00d	Q-03 Offline and disconnect resilience for Host (2.0d) PILOT-02 Squirrel or Omnivore confirmation plus the r.. (1.5d)	M-05 U shape diagnostic (1.5d) Q-01 Decision log audit view (2.0d) PILOT-01 Shadow mode runbook plus onboarding one real.. (2.0d)

e the other works. Totals stay exactly 50/50. The Sync cc

Sync points and handoffs this sprint

Tanner's F-01 and F-02 gate everything, so he front-loads F-07 and F-08 to unblock Kedi's roster and floor editor. Kedi runs the design system and own models in parallel, then starts I-01. This sprint leans Tanner because he is laying the scaffold, but Kedi is not idle.

Lowest coordination sprint. The engine lane (Tanner) and the ROI Internal lane (Kedi) are almost fully independent. Only link: E-03 reads the floor shape that I-03b produces, so order I-03b before E-03.

SHARED: X-01 SHARED SEAM

Settle X-01 (the floor channel contract) in writing before the views are built. Key handoff: Kedi's H-03 consumes Tanner's E-07 scoring output, so agree that payload early. C-01 and V-01 are Tanner's parallel filler so he is not idle while views are built.

SHARED: C-02 SHARED SEAM

Highest integration risk. C-02 (cut approval) is a shared seam spanning both surfaces. Handoffs: Tanner's P-02 consumes Kedi's R-01 (rush flag) and H-03, and Kedi's M-04 reads the pacing history P-01 produces, so order P-01 before M-04.

SHARED: Q-02 SHARED, Q-04 SHARED

Shared Q-02 (performance pass) and Q-04 (end to end night) are done together. Handoff: Tanner's Q-03 (host resilience) builds on Kedi's H-01b live binding. Lightest sprint, a buffer for hardening and pilot onboarding.